

Citationwalk: Network representation learning with scientific documents

Juhyun Lee^a, Sangsung Park^{b,*}, Junseok Lee^{c,*}^a Institute of Engineering Research, Korea University, Seoul 02841, South Korea^b Department of Data Science, Cheongju University, Cheongju 28503, South Korea^c Department of Data Science, Kangnam University, Youngin 16979, South Korea

ARTICLE INFO

Keywords:

Network

Representation learning

Node embedding

Link embedding

Document network

ABSTRACT

A network is a structure that can represent an organic relationship of observations. Network representation learning has the advantage of extracting latent features in a network. In recent years, various algorithms have been developed to embed network representation in a low-dimensional feature space. Conventional algorithms learn a sequence of vertices that are randomly searched from a network and convert a vertex into a vector. This study focused on the representation learning of document networks. Thus, this study proposes a scalable network representation learning algorithm that preserves the structure and content of the document network and reduces data sparsity. To do this, we used the shortest walks to extract a sequence of vertices that reflected semantic similarity from a network. The shortest walk is a method for finding several shortest paths that connect similar vertices. After this, the sequence of vertices was inputted into the language model, assuming that the sequence was a sentence. Scientific papers and patent datasets provided empirical evidence for the validity of our approach. The former, the bibliography network of scientific paper, consists of 17,716 nodes and 105,734 edges. The latter is a patent describing a technology for energy, representing a network of 24,894 nodes and 28,314 edges. The results showed that the proposed algorithm exhibited the lowest error of the tested methods in multi-class classification and link prediction tasks. The results of these tasks provided evidence that our algorithm is suitable for extracting feature representations of document networks.

1. Introduction

Bengio et al. (2013) defined representation learning as extracting hidden or entangled data features behind explanatory factors for machine learning. Representation learning has been widely used to discover latent patterns from unstructured data such as images. For example, autonomous driving technology employs images showing what is in front of the car, which are inputted by representation learning as data by segmenting images into roads or vehicles (Greff et al., 2019). Thus, representation learning can be a bridge that leads us into the next industrial revolution (Gourisaria et al., 2021; Paschek & Draghici, 2019).

Recently, in various fields such as computer engineering, networks have been used as the backbone of systems (Van Belle et al., 2022). A network is a method of representing objects as vertices or nodes and visualizing their relationships with links or edges. A network can visualize the features of objects via the sizes, shapes, colors, etc. of vertices. Links can represent a pattern among vertices using the widths of edges, etc. Furthermore, from a network of objects, the following can be

discovered: (i) what vertices are most connected to other vertices in a network of objects, (ii) what must pass through to go to other vertices, and (iii) what the shortest path to other vertices is.

Network representation learning (NRL) is a method of discovering latent features of data consisting of networks. Zhang et al. (2018) emphasized that there are several key challenges for achieving successful NRL. The first is structure preservation. Structure preservation aims to capture information on the similarity and distance between vertices to preserve the network structure. The second challenge is content preservation, which refers to the problem of preserving vertex attributes that contribute greatly to the formation of the network. This is because attributes are the basis required to measure the similarity between vertices. The third challenge is that data sparsity is related to the network data structure. Since a network summarizes objects and their relationships with vertices and links, it uses a larger and sparser matrix than other data. The final challenge is scalability, which refers to the reduction of the computational time cost in NRL algorithms. Thus, it is necessary to study NRL algorithms that reflect network structure and content and enable scalable computation from sparse data.

* Corresponding authors.

E-mail addresses: leeju@korea.ac.kr (J. Lee), hanyul@cju.ac.kr (S. Park), jxli12@kangnam.ac.kr (J. Lee).<https://doi.org/10.1016/j.eswa.2023.120372>

Received 13 November 2022; Received in revised form 21 March 2023; Accepted 1 May 2023

Available online 5 May 2023

0957-4174/© 2023 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

The approaches for NRL have been divided into the following forms: (i) matrix factorization (ii) random walk (iii) deep learning, and (iv) generative model-based methods. For the matrix factorization-based method, an algorithm that uses the modularity matrix or the vertex-context matrix after structuring network vertices into a matrix has been proposed (Y. Li et al., 2015; Tang & Liu, 2009; Yang et al., 2015). However, matrix factorization strategies have limitations in that they cannot capture the network's local structure and are computationally expensive.

A random walk is an algorithm that converts neighbors of vertices into a series of sequences (Fouss et al., 2007). Two of the representative random walk-based approaches are *DeepWalk* and *Node2Vec*, which treat a sequence of vertices extracted by a random walk as a sentence consisting of vocabularies, which are then used as the input for the language model (Grover & Leskovec, 2016; Perozzi et al., 2014). *Rank2Vec* is a method for embedding nodes by reflecting microscopic topological information and macroscopic role similarity (Zhou et al., 2019). They used *PageRank* to extract nodes' neighbors (Page et al., 1999). Recently, studies using not only nodes but also edge-based random walks have been conducted (Meilian & Danna, 2020). They generated node sequences with an edge-based dynamic random walk for representation learning on large networks. The random walk strategies can overcome the drawbacks of matrix factorization-based methods. However, they have the problem of the length of a sequence of vertices extracted from the network being static. Thus, it is necessary to create an approach in which the length of the walk can fluctuate according to the feature of vertices.

Given the improvement of computing power in recent years, conducting research on NRL methods using a neural network has been proposed (B. Li & Pi, 2020). Deep learning has the advantage of training nonlinear feature representation of networks. Cao et al. (2016), Fu et al. (2020), and D. Wang et al. (2016) have proposed deep neural network models for NRL. Their models can capture high-dimensional feature representation through deep learning. A generative adversarial network (GAN) is a deep learning model that can discriminate between true and fake distributions using a representative generative model (Goodfellow et al., 2020). GAN for NRL learns vertices and their underlying true distribution and discriminates fake vertices (Gao et al., 2019; H. Wang et al., 2018). Thus, GAN strategies can extract network representation by estimating the true distribution of vertices. However, the deep learning and generative model-based approaches have a limitation of time-consuming training.

Since a network is composed of vertices and links between objects, it requires a larger and sparser matrix than other data. Previous studies have proposed methods using matrix factorization, random walk, deep learning, and generative models for NRL. However, these methods are computationally expensive and intractable. Thus, we propose a random walk-based NRL method that can accurately capture a network structure with a low computational time cost. In particular, we employed the shortest walk to capture a structure of context, vertices, and links in a document network. We refer to this as the *CitationWalk*.

CitationWalk creates a sequence of vertices based on the similarity of documents. In this study, *CitationWalk* extracted a list of sequences with different lengths from a network using the shortest walk. We optimized the language model through the extracted sequence to maximize the likelihood of preserving structure and content. Our experiments focused on multi-class classification and link prediction tasks. The goal of the first task was to observe whether *CitationWalk* embedded a vertex in the low dimension to maintain similarity between documents, while the goal of the second task was to check whether *CitationWalk* could learn the feature representation of vertices and links. To achieve these goals, we conducted experiments using a large-scale network.

Overall, our paper makes the following contributions:

- We propose the shortest walk, which is a variation of a random walk, for the structure preservation of NRL.

- We used similarity between documents when extracting a sequence of vertices for content preservation of the network.
- For scalability, we reduced data sparsity through the shortest walk and lowered the time cost using a sequence of vertices that was smaller than the existing one.
- Since the length of the sequence of vertices in the shortest walk was dynamic, which is different from that of a random walk, the diversity of learning could be improved.

The rest of the paper is structured as follows. Section 2 provides the study background. Section 3 explains the proposed algorithm, and Section 4 describes the experiment. Finally, Section 5 discusses several limitations of our algorithm and draws suggestions for future works.

2. Background

A variety of information in documents can be represented by a network. In this section, (i) previous studies are investigated to analyze a document network, and (ii) methodologies of mapping network nodes into multi-dimensional space are described.

2.1. Document network analysis

A document is composed of a combination of multiple keywords. In addition, expert documents of scientific papers or patents are linked via a citation relationship. Previous studies have conducted predictive modeling or clustering analysis using the frequency of keywords as a variable (Han & Sohn, 2015; Jun et al., 2012; I. Park et al., 2020; Wu et al., 2021). In addition, citation information has been employed in various methodologies, being converted into the number of citations of documents (Meyer, 2001; Pagani et al., 2015; Y. Park et al., 2005). However, the text and citation of documents are sequential information. In other words, the key to two types of information is that their objects are connected and linked.

Calero-Medina and Noyons (2008) claimed that document citation is a method of evolution of the knowledge-construction process. This viewpoint has led to the development of document network analysis. Document network analysis refers to the conversion of information in documents into a network. Shibata et al. (2008) developed a method of detecting promising studies by creating a citation network of scientific documents. Huynh et al. (2012) proposed a citation network-based methodology for searching scientific articles whose context is similar. They improved the effectiveness of the algorithm by representing the relationship of documents with references as a network. Furthermore, Sterling and Montemore (2022) proposed a network-based system that can combine citations and texts of documents. Their system has the advantage of improving the problem of synonyms and multiple meanings in the keywords of documents.

With the development of various technologies in recent years, analysis of scientific documents has been actively conducted. This is because their citations reveal the development of science and innovation. Daim et al. (2020) proposed a methodology for forecasting technological positioning using a citation network. They predicted a trend in technology by using an indicator that can capture the knowledge status and reliability of scientific documents. In addition, Qiu and Wang (2020) applied citations as documents' text network to technology forecasting. As such, studies on the use of citation networks have gradually increased for prior art search, technology evaluation, and clustering (Higham et al., 2022; Huang et al., 2022; Lee et al., 2022; Mariani et al., 2019). However, previous studies have employed centrality, which refers to the relative importance of a node in a network as a variable. These approaches have a limitation in that they cannot represent features of observations in a multi-dimensional space. Thus, it is necessary to create a method that can learn features between nodes in a network.

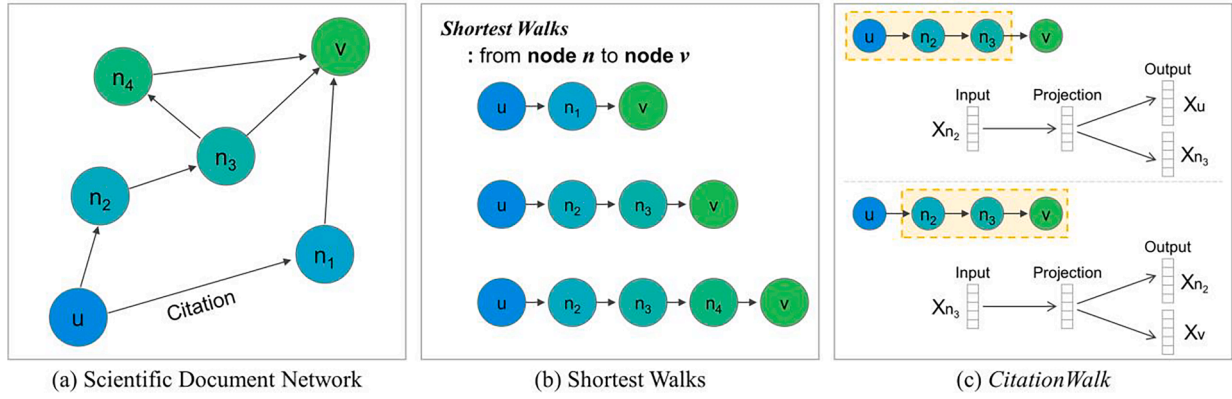


Fig. 1. Conceptual diagram of the proposed algorithm. (a) The concept of a network for scientific documents. (b) Concepts of shortest walks searched from the network. (c) The concept of the process of learning the feature representation of a node.

2.2. Node embedding

Node embedding is a method of learning the connection relationships between nodes in a network to represent their features in a multi-dimensional space. *DeepWalk* is an algorithm that learns the latent representation of nodes in a network (Perozzi et al., 2014). *DeepWalk* extracts the connection relationships of nodes from a network using a random walk. Then, it assumes a list of sequential nodes as a word in the sentence. Let network $G = (V, E)$, where V represents the vertex or node of the network, and E is their links or edges. Eq. (1) calculates the transition probability to extract truncated random walks, where the fixed length of walks is l when the source node is u . n_i denotes the i -th node in the walk ($n_0 = u$). The transition probability between nodes v and x , π_{vx} can be normalized using the normalizing constant C .

$$P(n_i = x | n_{i-1} = v) = \begin{cases} \frac{\pi_{vx}}{C} & \text{if } (v, x) \in E \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

DeepWalk learns the relationship between extracted nodes using *SkipGram*, which is a word embedding model. This model maximizes the probability of co-occurrence between neighbor words that appear within window w in a sentence. Let us assume that a sequence of nodes whose length extracted by a random walk from a network M is $v_1, v_2, \dots$, and v_M . Then, the objective function to be minimized by *SkipGram* is Eq. (2). In equation, $\log p(v_{m+n} | v_m)$ refers to the log probability of events where n words appear around the m -th word.

$$-\frac{1}{M} \sum_{m=1}^M \sum_{n=0, n \neq w}^{2w} \log p(v_{m+n} | v_m) \quad (2)$$

SkipGram embeds words with similar meanings by maximizing the probability of where a given keyword appears in a sentence. *DeepWalk* extracts γ walks whose length is l for each node using a random walk. It also embeds nodes in $\mathbb{R}^d$ after learning a sequence of nodes whose window size is w via *SkipGram*.

Node2Vec uses the second order random walk for more scalable feature learning than that of *DeepWalk* (Grover & Leskovec, 2016). Let us assume that a random walk is extracted from node u to nodes v and x . The unnormalized transition probability of *Node2Vec* for the second order random walk is calculated using Eq. (3):

$$\alpha_{pq}(v, x) = \begin{cases} \frac{1}{p} & \text{if } d_{vx} = 0 \\ 1 & \text{if } d_{vx} = 1 \\ \frac{1}{q} & \text{if } d_{vx} = 2 \end{cases} \quad (3)$$

where d_{vx} denotes the length of the shortest path between v and x . p and

q refer to the parameters that control the speed of exploring and leaving neighboring nodes, respectively. Note that the transition probability of the weighted network is the product of the edge's weight and $\alpha_{pq}(v, x)$.

In equation, p refers to the return parameter, which controls the likelihood of immediately revisiting a node in the walk. *Node2Vec* can use a small p value to learn the local information of nodes. The in-out parameter q allows the search to differentiate between "inward" and "outward" nodes. If q , which is smaller than 1, is used, *Node2Vec* can visit a far node to create a sequence of walks. Similar to *DeepWalk*, *Node2Vec* extracts γ walks whose fixed length is l for each node using the second order random walk and embeds nodes in $\mathbb{R}^d$ whose window size is w using *SkipGram*.

3. Feature learning framework

The purpose of this paper was to learn $X_v \in \mathbb{R}^{|V| \times d}$, where $|V|$ is the number of vertices, and d is a small number of dimensions of feature representations. Let $f: V \rightarrow X_v$ be the mapping function from the vertex to d -dimensions, which is called the *CitationWalk*. Thus, *CitationWalk* aims to learn for a downstream prediction task by training f .

Fig. 1 explains the conceptual diagram of *CitationWalk*. Fig. 1(a) shows the concept of a scientific document network. Nodes and edges in the network represent scientific documents and citations between them, respectively. Fig. 1(b) depicts the paths connecting nodes u and v extracted by shortest walks. Finally, *CitationWalk* learns the feature representations of nodes from the shortest walks, as shown in Fig. 1(c).

3.1. Shortest walks

Dijkstra (1959) proposed an algorithm that searched the shortest path that connected two nodes in a network. Dijkstra's algorithm differentiates nodes that were not visited among neighboring nodes and selects the shortest distance among them, and this process is iterated. Dijkstra's algorithm ensures that the shortest path between nodes is searched, even in an unweighted network (Thorup, 1999).

Yen (1970) proposed an algorithm that searches for γ shortest paths, including Dijkstra's shortest path, in a network. Yen's algorithm searches for γ paths from the source node u to the target node v . Let us assume that A and B are the searched shortest paths and a candidate set of shortest paths, respectively. Yen's algorithm searches Dijkstra's shortest paths from nodes u to v and uses the results to find paths. Note that P^k is the k -th shortest path among the paths from the source node to the target node. Eq. (4) presents the searched P^k path. n_i^k refers to the i -th node of P^k , i refers to a range from 1 to k , and $k+2$ refers to the P^k length (the reason for adding 2 is to include the source and target nodes). Thus, the Dijkstra's shortest path from nodes u to v is P^1 .

$$P^k = u \rightarrow n_1^k \rightarrow n_2^k \rightarrow \dots \rightarrow n_{i_k}^k \rightarrow v \quad (4)$$

Yen's algorithm searches R_i^k for all nodes except for u and v in the P^{k-1} path. R_i^k is the path of P_i^k that follows that P^{k-1} until the n_i^{k-1} . Next, previously visited paths are ignored, and S_i^k , which is the spur path of P_i^k that starts at the n_i^k and ends at the sink for each node, is searched. Then, P_i^k , which is the addition of each node of R_i^k and S_i^k , is added to Set B. Finally, the shortest paths in Set B are included in Set A and then deleted from set B. Yen's algorithm iterates and uses this process, thereby producing γ shortest walks that link u and v . The computational upper bound of shortest walks increases only linearly with the value of γ . We denote the shortest walk, which is the path from the i -th node v_i to j -th node v_j , as W_{ij} . If the number of shortest paths connecting v_i and v_j is less than or equal to γ , W_{ij} is skipped.

Suppose that search for the shortest walks connecting nodes u and v of Fig. 1(a). In Fig. 1(b), $u \rightarrow n_1 \rightarrow v$ describes the Dijkstra's shortest path of the two nodes. Next, shortest walks search $u \rightarrow n_2 \rightarrow n_3 \rightarrow v$, which is the shortest path longer than Dijkstra's path. This process helps *CitationWalk* learn various paths that connect scientific documents.

3.2. The CitationWalk algorithm

In this paper, paths that link two nodes through shortest walks were searched. Different paths that link the same pair of nodes can help *SkipGram* learn various representations. If γ is 1, the shortest walk returns the Dijkstra's shortest path between two nodes. The number of walks per vertex γ and the number of paths connecting two nodes tend to increase together. That is, walks per vertex γ is a hyperparameter of *CitationWalk*. Therefore, *CitationWalk* needs an appropriate γ according to the document type.

Algorithm 1 *CitationWalk*($G, w, d, \gamma, S, t, DATE$)

Input: network $G = (V, E)$

window size w
 embedding size d
 walks per vertex γ
 similarity matrix $S = \{s_{ij}\}$
 threshold of similarity t
 date vector $DATE = \{date_i\}$

Output: matrix of vertex representations $X \in \mathbb{R}^{|V| \times d}$

- 1: Initialization: Sample $X_V \in \mathbb{R}^{|V| \times d}$
- 2: **for** $i, j = 1$ to $|V|$ **do**
- 3: **if** $s_{ij} > t$ **do**
- 4: $TD = date_i - date_j$
- 5: **if** $TD \leq 0$ **then**
- 6: $v_E = v_i$ and $v_L = v_j$
- 7: **else**
- 8: $v_E = v_j$ and $v_L = v_i$
- 9: **end if**
- 10: **end if**
- 11: $W_{EL} = \text{shortestwalks}(G, v_E, v_L, \gamma)$
- 12: $\text{SkipGram}(X_V, W_{EL}, w)$
- 13: **end for**

Scientific documents include texts and citations. In this study, shortest walks were conducted for similar documents for efficient feature learning. The similarity matrix of $|V|$ documents is presented in Eq. (5):

$$S = \{s_{ij}\}, i, j = 1, 2, \dots, |V| \quad (5)$$

where element s_{ij} of similarity matrix S refers to a value of similarity level in the context of v_i and v_j measured by cosine similarity. Thus, the proposed method searches for the shortest walk of nodes v_i and v_j when s_{ij} is larger than the threshold t .

Citations in scientific documents tell us how science and technology have progressed. Let us assume that the publication date of v_E is earlier than that of v_L . Then, v_L can be cited by v_E . Thus, for the shortest walks, paths from v_E to v_L should be searched rather than those from v_L to v_E . The time difference TD between two documents is calculated by Eq. (6):

$$TD = date_i - date_j \quad (6)$$

where $date_i$ and $date_j$ refer to the publication date of v_i and v_j , respectively.

CitationWalk determines which document is published earlier using TD . If TD is equal to or smaller than 0, v_i is v_E and v_j is v_L . If TD is larger than 0, v_i is v_L and v_j is v_E . Thus, W_{EL} , which is the shortest walk of v_E and v_L , is the training data of *SkipGram*.

Algorithm 1 is the pseudo-code of *CitationWalk*. Window size w and embedding size d mean the number of neighboring nodes and the dimension of feature representations, respectively. Walks per vertex γ represent the number of shortest walks between v_E and v_L . Similarity t is a criterion for measuring similarity between two documents in s_{ij} . $DATE$ refers to the vector that has the publication dates $date_i$ of v_i as elements.

CitationWalk initializes X_V , which is the representation matrix of the vertices, following the uniform distribution. Next, a loop procedure is conducted for all nodes. Lines 3–11 in Algorithm 1 show the core of our approach. First, *CitationWalk* determines whether the two documents are similar. Then, for a pair of nodes that is determined to be similar, v_E and v_L are determined. Line 11 shows paths that connect v_E to v_L in the citation network G . If there is no path connecting v_E and v_L , skip and move on to the next vertices. Finally, *CitationWalk* updates the representation of W_{EL} with the *SkipGram* algorithm.

Fig. 1(c) depicts the feature representation of scientific documents obtained through *CitationWalk*. Our algorithm learns a number of shortest paths between node u and its neighbors. Then, X_u , the feature representation of node u , represents a vector whose embedding size is d .

3.3. Learning edge features

The purpose of the *CitationWalk* algorithm is to learn function $f: V \rightarrow X_V$, which embeds V in $\mathbb{R}^{|V| \times d}$ in the network $G = (V, E)$. That is, Algorithm 1 returns a feature representation of V through the citation information of similar documents. Then, we can embed E in the simple operator g . Eq. (7) presents the *Average* operator g_{Average} , which converts a mean of two nodes into the vector of E . In Eq. (7)–(10), $f_z(v_E)$ refers to the z -th element in the d -dimension embedding vector of node v_E .

$$f(v_E) \boxplus f(v_L) = \frac{1}{2} \times \{f_z(v_E) + f_z(v_L)\} \quad (7)$$

Eq. (8) presents the *Hadamard* operator g_{Hadamard} , which multiplies the elements of two nodes. The *Hadamard product* is a method for multiplying elements of a vector or matrix of the same size (Horn, 1990).

$$f(v_E) \boxtimes f(v_L) = f_z(v_E) \times f_z(v_L) \quad (8)$$

The *Weighted-L1* operator g_{L1} is presented in Eq. (9). This operator calculates the feature representation of edges using L1 regularization for the difference between elements.

$$\|f(v_E) \bullet f(v_L)\|_1 = |f_z(v_E) - f_z(v_L)| \quad (9)$$

The *Weighted-L2* operator g_{L2} , which uses L2 regularization of the difference between two elements, is presented in Eq. (10). The *Weighted-L2* operator returns the square of the element difference between two vectors. However, L2 regularization has the drawback of not being more robust to outliers than L1 regularization.

$$\|f(v_E) \bullet f(v_L)\|_2 = |f_z(v_E) - f_z(v_L)|^2 \quad (10)$$

Let us assume that edge E is $E \subseteq (V \times V)$ in network G . Let $f(u)$ and $f(v)$ be feature representations of nodes u and v , respectively. Then $g(u, v)$ denotes the feature representation of the edge connecting nodes u and v ($g(u, v) \in \mathbb{R}^d$).

Table 1

Parameters used for training each algorithm.

Parameters	CitationWalk	DeepWalk	Node2Vec	Node2Vec
embedding size d	128	128	128	128
walks per vertex γ	5	5	5	5
threshold of similarity t	0.6	—	—	—
walk length l	—	11	11	11
return parameter p	—	—	0.5	1.0
in-out parameter q	—	—	1.0	0.5

4. Experiments

Experiments were conducted on scientific documents. Journal or conference papers are typical scientific documents. In addition, the scientific paper aims to improve the limitations of previous studies. Therefore, the scientific paper cites a similar article. Patents are published documents that claim the rights to developed technologies. A patent cannot infringe upon the right of the prior art, which is a previously published document. Thus, a patent that cites prior art that is technically similar. We expect that the flow of scientific advancement can be reflected in feature representation through the *CitationWalk* algorithm.

4.1. Experimental setup

The purpose of *CitationWalk* proposed in this paper is to extract feature representations from scientific document networks. The experiment used two types of scientific documents. We utilized the following datasets:

- DBLP (Bojchevski & Günnemann, 2017; Pan et al., 2016): DBLP is bibliography data for scientific papers. The network has 17,716 nodes, 105,734 edges, and 4 different labels (Y_1^D to Y_4^D).
- PATENT: We have collected patents on energy technology. The labels represent technology inferred through contextual similarity between documents (see appendix A). The network has 24,894 nodes, 28,314 edges, and 6 different labels (Y_1^P to Y_6^P).

We extracted the similarity between scientific documents and feature representation of citations through experiments. In the experiments, the performance of the proposed algorithm was compared with those of several algorithms to evaluate the proposed algorithm. The first algorithm was *DeepWalk*. The second algorithm was *Node2Vec* whose p and q were 0.5 and 1.0, respectively. The final algorithm was *Node2Vec* whose p and q were 1.0 and 0.5, respectively. The parameters of the algorithms used in the experiments are presented in Table 1. *CitationWalk* where the length of walk per vertex is dynamic, has different window sizes depending on the dataset. Therefore, in DBLP, we set the window size to 3 for all algorithms. In the patent dataset, we set the window sizes of *CitationWalk* and other models to 11 and 5, respectively.

4.2. Multi-class classification

CitationWalk embeds network nodes after learning the similarity and citation information of documents. Thus, we aimed to compare the performance of the proposed algorithm in classification according to the document context. First, we obtain the feature representation of a node through the NRL algorithm. Then, we predict the label with the node's

Table 2

Comparison of performance measures by NRL algorithms.

Dataset	Algorithm	Accuracy	Macro Precision	Macro Recall	Macro F1-score
DBLP	<i>CitationWalk</i>	0.800	0.755	0.739	0.746
	<i>DeepWalk</i>	0.567	0.440	0.381	0.389
	<i>Node2Vec</i> ($p = 0.5, q = 1.0$)	0.568	0.436	0.382	0.390
	<i>Node2Vec</i> ($p = 1.0, q = 0.5$)	0.562	0.432	0.375	0.383
PATENT	<i>CitationWalk</i>	0.293	0.210	0.220	0.195
	<i>DeepWalk</i>	0.271	0.188	0.194	0.167
	<i>Node2Vec</i> ($p = 0.5, q = 1.0$)	0.261	0.176	0.192	0.169
	<i>Node2Vec</i> ($p = 1.0, q = 0.5$)	0.250	0.195	0.191	0.174

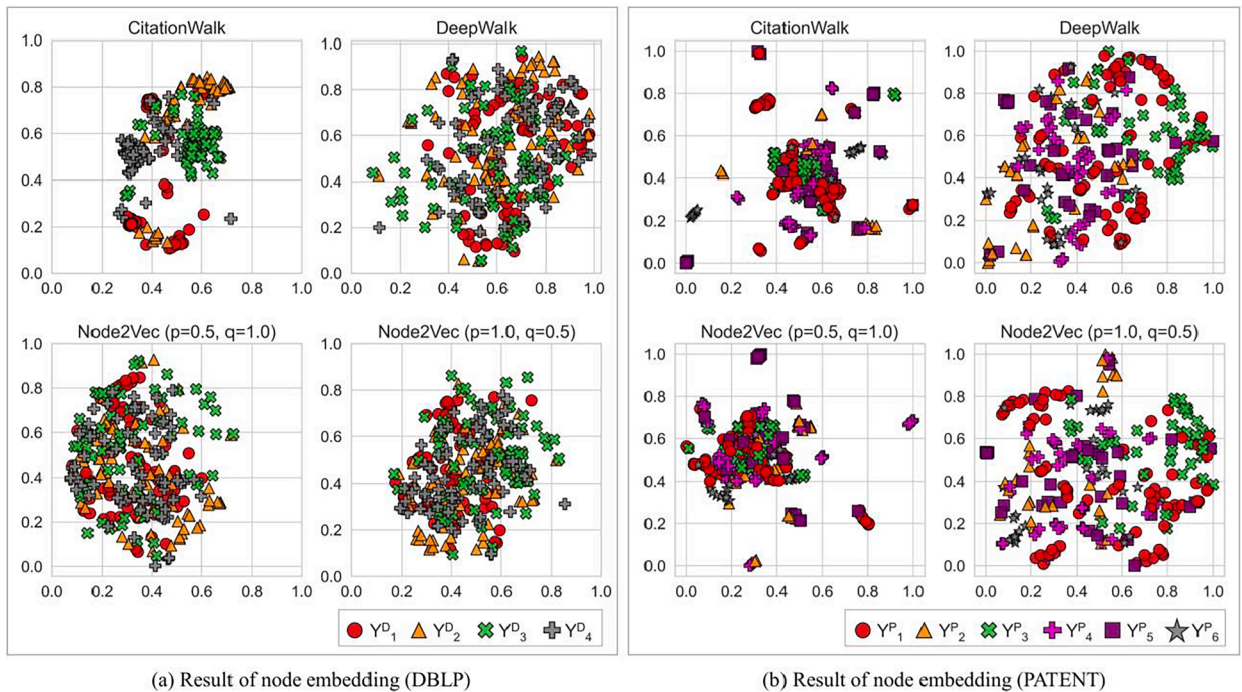


Fig. 2. Visualization of node embeddings according to the dataset. (a) Visualization of the DBLP dataset according to the NRL algorithm. (b) Visualization of the PATENT dataset according to the NRL algorithm.

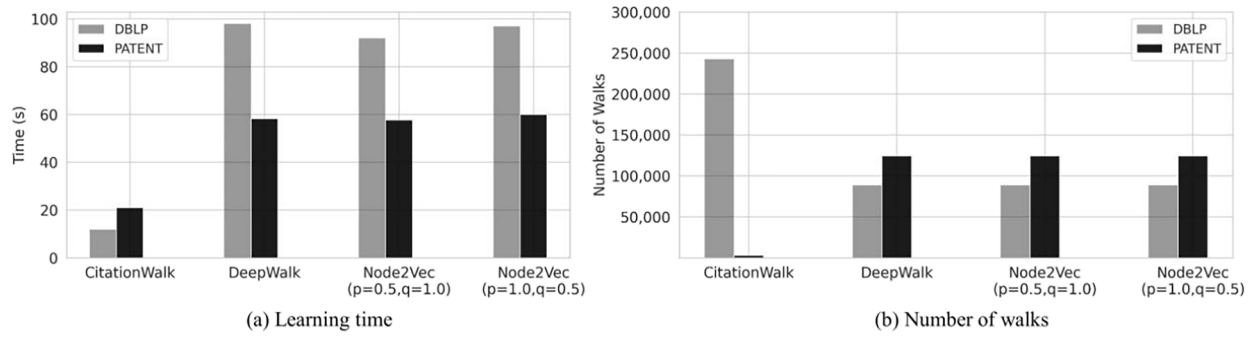


Fig. 3. Comparison of learning time and number of walks. (a) The learning time of each algorithm. (b) The number of walks of each algorithm.

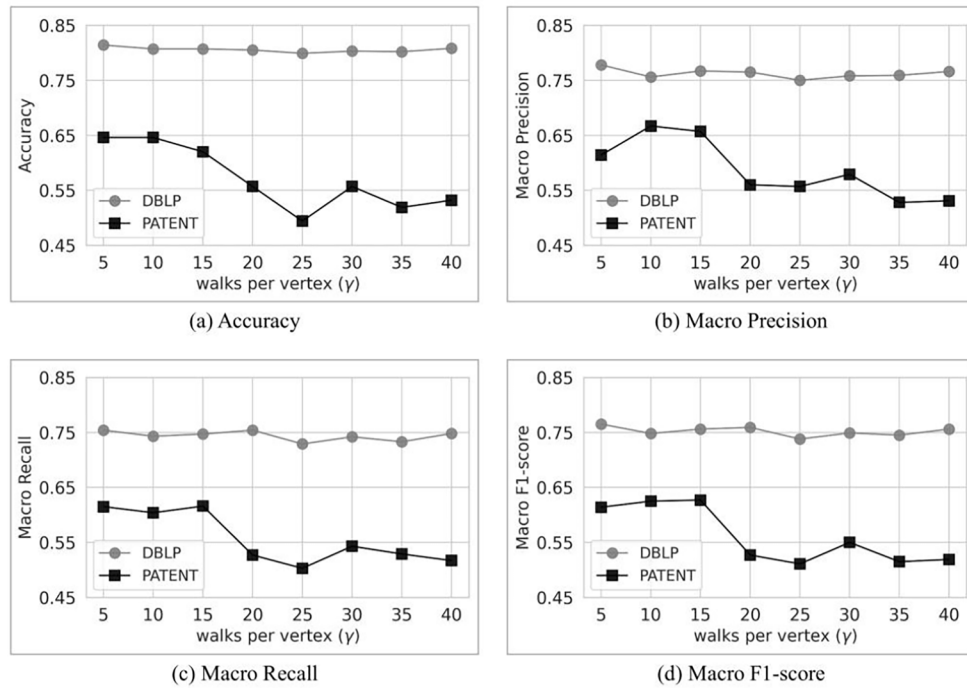


Fig. 4. Results of hyperparameter ablation trial. (a) Accuracy according to walks per vertex. (b) Macro precision. (c) Macro recall. (d) Macro f1-score.

Table 3
Comparison of AUC scores for link prediction.

Operator	Algorithm	Dataset DBLP	PATENT
Average	DeepWalk	0.731	0.718
	Node2Vec($p = 0.5, q = 1.0$)	0.749	0.718
	Node2Vec($p = 1.0, q = 0.5$)	0.753	0.755
	CitationWalk	0.988	0.737
Hadamard	DeepWalk	0.770	0.694
	Node2Vec($p = 0.5, q = 1.0$)	0.773	0.694
	Node2Vec($p = 1.0, q = 0.5$)	0.793	0.720
	CitationWalk	0.992	0.743
Weighted-L1	DeepWalk	0.724	0.739
	Node2Vec($p = 0.5, q = 1.0$)	0.742	0.739
	Node2Vec($p = 1.0, q = 0.5$)	0.744	0.740
	CitationWalk	0.974	0.715
Weighted-L2	DeepWalk	0.714	0.716
	Node2Vec($p = 0.5, q = 1.0$)	0.735	0.716
	Node2Vec($p = 1.0, q = 0.5$)	0.738	0.745
	CitationWalk	0.971	0.697

representation vector (McInnes et al., 2020). Fig. 2(a) depicts the distribution of labels for the DBLP dataset. The red represents Y_1^D , orange represents Y_2^D , green represents Y_3^D , and gray represents Y_4^D . Compared to other NRL algorithms, CitationWalk returns vectors separated by labels of scientific papers. Fig. 2(b) describes the distribution of labels for the PATENT dataset. The red represents Y_1^P , orange represents Y_2^P , green represents Y_3^P , pink represents Y_4^P , purple represents Y_5^P , and the gray represents Y_6^P . We can see that CitationWalk distinguishes well according to the contextual similarity of technologies.

Fig. 2(a) and (b) show how well CitationWalk distinguishes representation features according to contextual similarity of scientific documents. Compared to other algorithms, CitationWalk embeds scientific documents with high contextual similarity very closely. On the other hand, the results of other algorithms show a uniform distribution regardless of similarity. How well does CitationWalk retain contextual similarities? To answer this question, the multi-class classifies the labels of scientific documents.

Table 2 presents the results of multi-class classification according to document context. The used classifier was a k-nearest neighbor (k-NN). The performance measures we used were accuracy, macro precision, macro recall, and macro f1-score. The classification performance was measured by changing the ratio of training and test data from 0.1 to 0.9

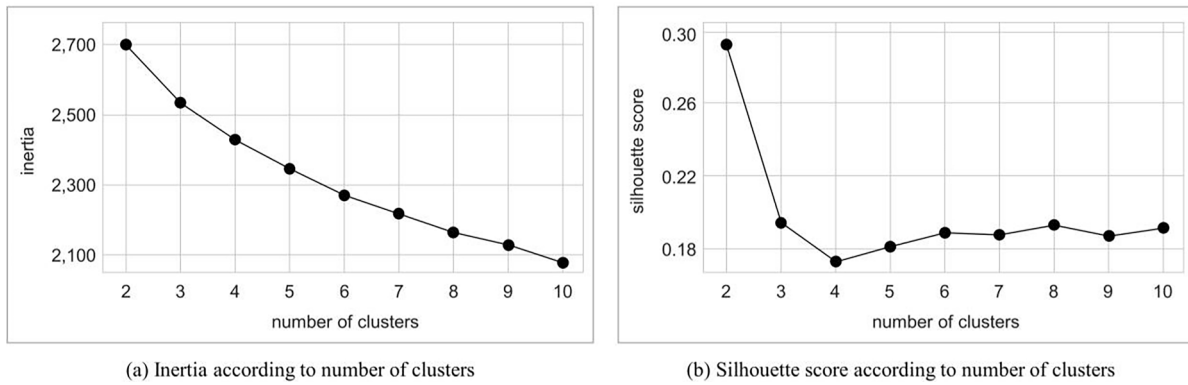


Fig. A1. Experimental results to determine the label of the PATENT dataset. (a) Inertia according to number of clusters. (b) Silhouette score according to number of clusters.

Table B1

Comparison of performance measures for DBLP dataset by NRL algorithms.

Measure	Algorithm	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	Avg.
Accuracy	<i>DeepWalk</i>	0.570	0.571	0.578	0.579	0.570	0.574	0.571	0.558	0.532	0.567
	<i>Node2Vec</i> ($p = 0.5, q = 1.0$)	0.578	0.575	0.582	0.565	0.572	0.566	0.565	0.556	0.550	0.568
	<i>Node2Vec</i> ($p = 1.0, q = 0.5$)	0.562	0.568	0.573	0.566	0.566	0.560	0.559	0.562	0.546	0.562
	<i>CitationWalk</i>	0.813	0.789	0.807	0.803	0.803	0.803	0.803	0.792	0.783	0.800
Macro Precision	<i>DeepWalk</i>	0.426	0.466	0.457	0.465	0.454	0.441	0.438	0.426	0.389	0.440
	<i>Node2Vec</i> ($p = 0.5, q = 1.0$)	0.446	0.444	0.463	0.434	0.455	0.439	0.426	0.405	0.415	0.436
	<i>Node2Vec</i> ($p = 1.0, q = 0.5$)	0.452	0.461	0.456	0.435	0.439	0.418	0.412	0.427	0.389	0.432
	<i>CitationWalk</i>	0.786	0.744	0.763	0.757	0.752	0.754	0.757	0.740	0.741	0.755
Macro Recall	<i>DeepWalk</i>	0.384	0.402	0.396	0.397	0.392	0.385	0.373	0.363	0.333	0.381
	<i>Node2Vec</i> ($p = 0.5, q = 1.0$)	0.403	0.389	0.396	0.380	0.396	0.378	0.371	0.367	0.358	0.382
	<i>Node2Vec</i> ($p = 1.0, q = 0.5$)	0.387	0.391	0.393	0.382	0.389	0.364	0.368	0.360	0.344	0.375
	<i>CitationWalk</i>	0.759	0.719	0.752	0.749	0.744	0.745	0.739	0.733	0.712	0.739
Macro F1-score	<i>DeepWalk</i>	0.390	0.414	0.408	0.408	0.404	0.394	0.380	0.370	0.336	0.389
	<i>Node2Vec</i> ($p = 0.5, q = 1.0$)	0.411	0.399	0.407	0.389	0.408	0.387	0.377	0.370	0.365	0.390
	<i>Node2Vec</i> ($p = 1.0, q = 0.5$)	0.398	0.404	0.405	0.391	0.398	0.368	0.372	0.365	0.344	0.383
	<i>CitationWalk</i>	0.771	0.731	0.757	0.753	0.748	0.749	0.747	0.736	0.724	0.746

Table B2

Comparison of performance measures for PATENT dataset by NRL algorithms.

Measure	Algorithm	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	Avg.
Accuracy	<i>DeepWalk</i>	0.251	0.234	0.284	0.280	0.244	0.276	0.253	0.245	0.370	0.271
	<i>Node2Vec</i> ($p = 0.5, q = 1.0$)	0.251	0.254	0.235	0.319	0.229	0.257	0.279	0.264	0.259	0.261
	<i>Node2Vec</i> ($p = 1.0, q = 0.5$)	0.226	0.215	0.295	0.236	0.298	0.238	0.279	0.245	0.222	0.250
	<i>CitationWalk</i>	0.251	0.220	0.240	0.299	0.321	0.295	0.354	0.245	0.407	0.293
Macro Precision	<i>DeepWalk</i>	0.099	0.181	0.252	0.221	0.176	0.203	0.161	0.112	0.291	0.188
	<i>Node2Vec</i> ($p = 0.5, q = 1.0$)	0.173	0.164	0.166	0.250	0.150	0.163	0.224	0.134	0.164	0.176
	<i>Node2Vec</i> ($p = 1.0, q = 0.5$)	0.166	0.129	0.222	0.209	0.287	0.219	0.171	0.201	0.147	0.195
	<i>CitationWalk</i>	0.140	0.149	0.127	0.280	0.238	0.240	0.236	0.164	0.319	0.210
Macro Recall	<i>DeepWalk</i>	0.167	0.183	0.225	0.198	0.175	0.205	0.180	0.161	0.254	0.194
	<i>Node2Vec</i> ($p = 0.5, q = 1.0$)	0.195	0.170	0.170	0.250	0.170	0.182	0.211	0.183	0.193	0.192
	<i>Node2Vec</i> ($p = 1.0, q = 0.5$)	0.173	0.156	0.212	0.188	0.221	0.184	0.211	0.189	0.185	0.191
	<i>CitationWalk</i>	0.171	0.178	0.178	0.212	0.236	0.236	0.266	0.185	0.318	0.220
Macro F1-score	<i>DeepWalk</i>	0.117	0.174	0.189	0.177	0.153	0.188	0.159	0.124	0.221	0.167
	<i>Node2Vec</i> ($p = 0.5, q = 1.0$)	0.180	0.150	0.151	0.236	0.155	0.158	0.193	0.147	0.154	0.169
	<i>Node2Vec</i> ($p = 1.0, q = 0.5$)	0.162	0.133	0.187	0.183	0.204	0.180	0.181	0.174	0.162	0.174
	<i>CitationWalk</i>	0.132	0.157	0.142	0.187	0.215	0.218	0.238	0.166	0.297	0.195

to compare the robustness of the node embedding algorithm. Appendix B describes the performance according to the ratio. The performance in the table represents the average according to the ratio. On the DBLP dataset, the accuracy of *CitationWalk* is 0.800, which is very high compared to other algorithms. Similarly, the other measures of *CitationWalk* were very high at 0.755, 0.739, and 0.746, respectively. On the PATENT dataset, the average performance of *CitationWalk* is insignificant compared to other algorithms. However, the proposed algorithm performed the best on the PATENT dataset.

Fig. 3(a) shows a graph that compares the learning times of the

algorithms. The learning time for *CitationWalk* was 11.9 s and 21 s, respectively, on DBLP and PATENT datasets, which were shorter than other algorithms. Fig. 3(b) depicts the number of walks used for training the NRL algorithm. In the DBLP dataset, the number of walks of *CitationWalk* was 242,820. The number of walks of *DeepWalk* and *Node2Vec* was 88,580, which was less than the proposed algorithm. Conversely, the number of walks in the PATENT dataset is 3,152. In addition, the number of walks of other algorithms is 124,470. That is, the number of walks of the proposed algorithm was the lowest. We pointed to the edges of the dataset as the cause of this result. This is because there are more

edges of DBLP than edges of PATENT. Therefore, we expect *CitationWalk* to explore more walks in networks with many edges.

The multi-class classification performance of the node embedding algorithm was measured by comparing how well the document context was maintained. The measurement results showed that *CitationWalk* had the best classification performance. In *CitationWalk*, the number of walks used for training increases with the edges of the network. Nonetheless, *CitationWalk* extracted the features of nodes in a short time. Thus, *CitationWalk* was deemed appropriate for node embedding tasks due to its reflection of the similarity and citations of scientific documents.

4.3. Hyperparameter ablation trial

CitationWalk contains walks per vertex γ as its hyperparameter. We observe the ablation trial, as the hyperparameters fluctuate. The procedure of the hyperparameter ablation trial is as follows. First of all, the proposed algorithm learns the DBLP and PATENT datasets while changing walks per vertex γ . Then, we validate the performance of the learned algorithm through multi-class classification. In the hyperparameter ablation trial, the range of used γ was $\{5, 10, 15, 20, 25, 30, 35, 40\}$.

Fig. 4(a)-(d) show the results of the hyperparameter ablation trial according to the dataset. Fig. 4(a) is the accuracy of *CitationWalk* according to γ . In our approach, the accuracy of multi-class classification decreased as the walks per vertex γ increased. In particular, the range of variation was large in the PATENT dataset. Macro precision, recall, and f1-score showed the highest performance when the hyperparameter was between 5 and 15. When the hyperparameter was greater than 20, the classification performance decreased rapidly.

What we can learn from trying ablation trial is as follow:

- The shortest walk searches more paths as walks per vertex γ increase. This gives *CitationWalk* an opportunity to learn overfitted features.
- The smaller the number of edges in the network, the greater the influence of walks per vertex. In the ablation trial, the PATENT dataset has fewer edges than DBLP. In this case, many walks per vertex can select redundant edges.
- In networks with many edges, the hyperparameters of *CitationWalk* are robust. In particular, the DBLP dataset shows little variation in performance according to the change of γ . Therefore, we need to determine the appropriate walks per vertex according to the number of edges.

4.4. Link prediction

Link embedding is the task of converting embedding f of two nodes into various operators through *CitationWalk*. We conducted link prediction using data where the prediction variable was $g: V \times V \rightarrow X_E$. The purpose of this experiment was to verify whether the link embedding of *CitationWalk* that was converted into operator g could maintain the document context well. To do this, we performed link prediction tasks. First, the link prediction converted the embedding vector of two nodes into operator g . Second, the target variable is the edges and non-edges of nodes. That is, link prediction is a binary-class classification task. Finally, we measure the performance of link prediction. As by convention, we measure the performance of link prediction by the area under the receiver operating characteristic curve (AUC).

Table 3 describes the results of link prediction according to operators and algorithms. In the DBLP dataset, *CitationWalk* had a very high AUC score of 0.992 compared to other algorithms. In particular, the AUC score was the highest when the *Hadamard* operator was used. On the other hand, the PATENT dataset showed little variation in performance depending on the operator. Nevertheless, when learning edge features with the *Hadamard* operator, *CitationWalk*'s AUC score was the highest at 0.743. This means that *CitationWalk* is suitable for learning features of

nodes and edges from a network of scientific documents.

5. Conclusions

5.1. Discussion and implications

Representation learning is a method for extracting features of unstructured data. As real-world data have been linked organically in recent years, network structure-based approaches have increased. NRL has an advantage in that it can project complex causal relationships of samples into a low-dimensional space. Previous studies have employed matrix factorization, random walk, deep learning, and generative model-based methods for NRL. However, most algorithms are computationally expensive and intractable due to network data sparsity. Thus, this study proposes *CitationWalk*, which is a scalable algorithm that preserves the network structure and content and reduces data sparsity through shortest walks.

This paper raises the question of how well the proposed algorithm can capture the features of nodes and edges from scientific document networks. In order to answer this question, we conducted experiments with scientific paper and patent datasets. The scientific paper and patent datasets are DBLP and PATENT, respectively. The DBLP dataset is a bibliography data for scientific papers. PATENT is patent data for energy technology. The network for DBLP consists of 17,716 nodes and 105,734 edges. The network for PATENT consists of 24,894 nodes and 28,314 edges. In the experiments, we tried to do tasks that we considered the most valuable to the NRL algorithms: (i) multi-class classification and (ii) link prediction.

The purpose of multi-class classification is to monitor how well *CitationWalk* retains the contextual similarities of scientific documents. The labels of DBLP and PATENT represent categories according to the contextual similarity of scientific documents. In the multi-class classification, the accuracy of *CitationWalk* was the highest at 0.800 and 0.293 for the DBLP and PATENT datasets, respectively. Through this, we were able to find empirical evidence that *CitationWalk* is suitable for representation learning of scientific document networks.

The purpose of link prediction is to check how well the NRL algorithm retains the edge features. Edge features were obtained using *Average*, *Hadamard*, *Weighted-L1*, and *Weighted-L2* operators. In an experiment to classify edges and non-edges, *CitationWalk* using the *Hadamard* operator retained the edge features best. Finally, the hyperparameter ablation trial explained the influence of the hyperparameters of the proposed algorithm. When the edges of the network were small, *CitationWalk*'s performance deteriorated as it learned duplicated sequences of nodes. In networks with many edges, *CitationWalk* showed robust performance according to walks per vertex. Through the ablation trial, we learned that an appropriate hyperparameter should be selected according to the type of scientific document.

5.2. Limitations and further research

This study focused on representation learning of document networks. Documents of scientific papers or patents can be viewed as vertices that are connected through similarity and citation. However, existing document network analysis has only used the number of citations or centrality of documents. We proposed *CitationWalk* to extract feature representation in a document network. First, *CitationWalk* calculates the similarity between two documents. Then, it compares their publication dates. The shortest walk searches for paths connecting recently published scientific documents from the past. The number of searched paths is γ . In addition, *SkipGram* conducts learning to maximize the likelihood of preserving the structure and content.

This paper presented empirical evidence that *CitationWalk* is suitable for NRL of scientific documents as an experiment on scientific paper and patent dataset. In particular, multi-class classification and link prediction solved the question of whether *CitationWalk* could well maintain

contextual similarities of scientific documents.

Nonetheless, the limitations of *CitationWalk* are as follows:

- *CitationWalk* considers the temporal factors of documents. However, we could not measure the results over the temporal range of scientific documents. Obviously, the publication date of a scientific paper or patent will reflect the flow of science and innovation.
- *CitationWalk* maximized the likelihood of preserving the structure and content using *SkipGram*. However, we did not apply a sequence of vertices for various language models (Bojanowski et al., 2017; Pennington et al., 2014; Peters et al., 2018).
- *CitationWalk* is an NRL algorithm that is optimized for document networks. However, we did not present a method that can be applied to other types of data for universal use.

The present study proposed *CitationWalk*, which is a novel algorithm for feature representation learning in a scientific document network. Future studies need to overcome the following limitations. First, future studies should discuss in detail the impact of the date a scientific document is published. Depending on the scope of the published date, the results of NRL will be different. Second, advanced NRL algorithms are needed to examine their performances using various types of language models. Finally, NRL algorithms need to be expanded to apply to other unstructured data such as images or signals. Through this, we expect more universal NRL algorithms to be developed.

Author contributions.

Juhyun Lee designed the algorithm, implemented the model and co-

authored the draft. Sangsung Park reviewed the manuscript and gave valuable advacies on how to improve it. Junseok Lee co-authored the draft and revise the manuscript very carefully. All authors have read and agreed to the published version of the manuscripts.

CRedit authorship contribution statement

Juhyun Lee: Methodology, Software, Visualization, Writing – original draft. **Sangsung Park:** Project administration, Writing – review & editing. **Junseok Lee:** Project administration, Writing – review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This research was supported by Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education(No. NRF-2022R111A1A01069422).

Appendix

A. Detailed description of dataset

DBLP is a representative benchmark dataset for a bibliography of scientific papers (Bojchevski & Günnemann, 2017). The labels of the DBLP dataset, Y_1^D to Y_4^D , indicate categories according to the contextual similarity of scientific papers (Pan et al., 2016). We use patent, a representative scientific document, for experiments. However, the PATENT dataset only includes text and citation relationships. Thus, we use the cluster according to the contextual similarity of technology as a label.

The process for labeling the PATENT dataset is as follows. First, we embed the PATENT dataset into a vector space as a pre-trained language model (Reimers & Gurevych, 2019). Second, the distance between the embedded document vectors is measured and clusters are assigned using k-means clustering. Finally, the cluster assigned to the document is used as the label of the NRL. We determined the number of clusters for the PATENT dataset by inertia and silhouette scores. Inertia represents the sum of squares of distances between the center of a cluster and observations belonging to the cluster. Inertia decreases as observations within a cluster are more similar. The silhouette score refers to the similarity of observations belonging to a cluster. The more similar the observations within a cluster, the higher the silhouette score.

Fig. A1(a) shows the inertia according to the number of clusters. Inertia gradually decreased as the number of clusters increased. Fig. A1(b) depicts the silhouette score according to the number of clusters. The silhouette score increased from 4 to 6 in the number of clusters, and there was no significant change thereafter. Therefore, this paper determined the number of labels for the PATENT dataset to be 6 (Y_1^P to Y_6^P).

Appendix B. .

Comparison of performance according to the dataset.

Table B1

Table B2

References

- Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8), 1798–1828. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6472238>.
- Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics*, 5, 135–146. https://doi.org/10.1162/tacl_a_00051/1567442/tacl_a_00051.pdf
- Bojchevski, A., & Günnemann, S. (2017). Deep Gaussian Embedding of Graphs: Unsupervised Inductive Learning via Ranking. *arXiv:1707.03815*.
- Calero-Medina, C., & Noyons, E. C. M. (2008). Combining mapping and citation network analysis for a better understanding of the scientific development: The case of the absorptive capacity field. *Journal of Informetrics*, 2(4), 272–279. <https://doi.org/10.1016/J.JOI.2008.09.005>
- Cao, S., Lu, W., & Xu, Q. (2016). Deep Neural Networks for Learning Graph Representations. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 1145–1152. <https://doi.org/10.1609/AAAI.V30I1.10179>.
- Daim, T., Lai, K. K., Yalcin, H., Alsoubie, F., & Kumar, V. (2020). Forecasting technological positioning through technology knowledge redundancy: Patent citation analysis of IoT, cybersecurity, and Blockchain. *Technological Forecasting and Social Change*, 161, Article 120329. <https://doi.org/10.1016/J.TECHFORE.2020.120329>
- Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1(1), 269–271.
- Fouss, F., Pirotte, A., Renders, J., & Saelens, M. (2007). Random-walk computation of similarities between nodes of a graph with application to collaborative

- recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 19(3), 355–369. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4072747>.
- Fu, S., Liu, W., Tao, D., Zhou, Y., & Nie, L. (2020). HesGCN: Hessian graph convolutional networks for semi-supervised classification. *Information Sciences*, 514, 98. <https://doi.org/10.1016/j.ins.2019.11.019>
- Gao, H., Pei, J., & Huang, H. (2019). ProGAN: Network Embedding via Proximity Generative Adversarial Network. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1308–1316. <https://doi.org/10.1145/1122445.1122456>.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... Bengio, Y. (2020). Generative adversarial networks. *Communications of the ACM*, 63(11), 139–144. <https://doi.org/10.1145/3422622>
- Gourisaria, M. K., Agrawal, R., Harshvardhan, G., Pandey, M., & Rautaray, S. S. (2021). Application of Machine Learning in Industry 4.0. In *In Machine Learning: Theoretical Foundations and Practical Applications* (Vol. 87, pp. 57–87). Springer Science and Business Media Deutschland GmbH. https://doi.org/10.1007/978-981-33-6518-6_4/COVER.
- Greff, K., Kaufman, R. L., Kabra, R., Watters, N., Burgess, C., Zoran, D., ... Lerchner, A. (2019). Multi-Object Representation Learning with Iterative Variational Inference. *International Conference on Machine Learning PMLR*, 2424–2433. <https://proceedings.mlr.press/v97/greff19a.html>.
- Grover, A., & Leskovec, J. (2016). node2vec: Scalable Feature Learning for Networks. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 855–864. <https://doi.org/10.1145/2939672>.
- Han, E., & Sohn, S. (2015). Patent valuation based on text mining and survival analysis. *Journal of Technology Transfer*, 40(5), 821–839. <https://doi.org/10.1007/S10961-014-9367-6/TABLES/10>
- Higham, K., Contisciani, M., & De Bacco, C. (2022). Multilayer patent citation networks: A comprehensive analytical framework for studying explicit technological relationships. *Technological Forecasting and Social Change*, 179, Article 121628. <https://doi.org/10.1016/j.techfore.2022.121628>
- Horn, R. A. (1990). The Hadamard product. In *Proceedings of Symposia in Applied Mathematics*, 87–169. <https://doi.org/10.1090/PSAPM/040/1059485>
- Huang, Y., Li, R., Zou, F., Jiang, L., Porter, A. L., & Zhang, L. (2022). Technology life cycle analysis: From the dynamic perspective of patent citation networks. *Technological Forecasting and Social Change*, 181, Article 121760. <https://doi.org/10.1016/j.techfore.2022.121760>
- Huynh, T., Hoang, K., Do, L., Tran, H., Luong, H., & Gauch, S. (2012). Scientific publication recommendations based on collaborative citation networks. *Proceedings of the 2012 International Conference on Collaboration Technologies and Systems, CTS 2012*, 316–321. <https://doi.org/10.1109/CTS.2012.6261069>.
- Jun, S., Park, S., & Jang, D. (2012). Technology forecasting using matrix map and patent clustering. *Industrial Management and Data Systems*, 112(5), 786–807. <https://doi.org/10.1108/02635571211232352/FULL/XML>
- Lee, J., Park, S., & Lee, J. (2022). A Fast and Scalable Algorithm for Prior Art Search. *IEEE Access*, 10, 7396–7407. <https://doi.org/10.1109/ACCESS.2022.3141494>
- Li, B., & Pi, D. (2020). Network representation learning: A systematic literature review. *Neural Computing and Applications*, 32(21), 16647–16679. <https://doi.org/10.1007/s00521-020-04908-5>
- Li, Y., Xu, L., Tian, F., Jiang, L., Zhong, X., & Chen, E. (2015, June 27). Word Embedding Revisited: A New Representation Learning and Explicit Matrix Factorization Perspective. *Twenty-Fourth International Joint Conference on Artificial Intelligence*. <https://www.aaai.org/ocs/index.php/IJCAI/IJCAI15/paper/view/10863>.
- Mariani, M. S., Medo, M., & Lafond, F. (2019). Early identification of important patents: Design and validation of citation network metrics. *Technological Forecasting and Social Change*, 146, 644–654. <https://doi.org/10.1016/j.techfore.2018.01.036>
- McInnes, L., Healy, J., & Melville, J. (2020). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *arXiv:1802.03426*.
- Meilian, L. U., & Danna, Y. E. (2020). HIN_DRL: A random walk based dynamic network representation learning method for heterogeneous information networks. *Expert Systems With Applications*, 158, Article 113427. <https://doi.org/10.1016/j.eswa.2020.113427>
- Meyer, M. S. (2001). Patent citation analysis in a novel field of technology: An exploration of nano-science and nano-technology. *Scientometrics*, 51(1), 163–183.
- Pagani, R. N., Kovaleski, J. L., & Resende, L. M. (2015). Methodi Ordinatio: A proposed methodology to select and rank relevant scientific papers encompassing the impact factor, number of citation, and year of publication. *Scientometrics*, 105(3), 2109–2135. <https://doi.org/10.1007/S11192-015-1744-X/TABLES/3>
- Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). The PageRank citation ranking: Bringing order to the web. *Stanford InfoLab*. <https://doi.org/10.1109/IISWC.2012.6402911>
- Pan, S., Wu, J., Zhu, X., Zhang, C., & Wang, Y. (2016). Tri-Party Deep Network Representation. *Network*, 11(9), 12.
- Park, I., Kim, S., & Yoon, B. (2020). Technology Clustering Using Textual Information of Reference Titles in Scientific Paper. *Journal of the Society of Korea Industrial and Systems Engineering*, 43(2), 25–32. <https://doi.org/10.11627/jkise.2020.43.2.025>.
- Park, Y., Yoon, B., & Lee, S. (2005). The idiosyncrasy and dynamism of technological innovation across industries: Patent citation analysis. *Technology in Society*, 27(4), 471–485. <https://doi.org/10.1016/J.TECHSOC.2005.08.003>
- Paschek, D., & Draghici, A. (2019). Industry 5.0—The expected impact of next industrial revolution. In *Thriving on Future Education, Industry, Business, and Society, Proceedings of the MakeLearn and TIIM International Conference*, 15–17.
- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532–1543.
- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 701–710. <https://doi.org/10.1145/2623330.2623732>.
- Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., & Zettlemoyer, L. (2018). Deep contextualized word representations. *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2227–2237. <https://doi.org/10.18653/v1/N18-1202>.
- Qiu, Z., & Wang, Z. (2020). Technology forecasting based on semantic and citation analysis of patents: A case of robotics domain. *IEEE Transactions on Engineering Management*, 69(4), 1216–1236. <https://doi.org/10.1109/TEM.2020.2978849>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *ArXiv:1908.10084*. <https://doi.org/10.48550/arxiv.1908.10084>.
- Shibata, N., Kajikawa, Y., Takeda, Y., & Matsushima, K. (2008). Detecting emerging research fronts based on topological measures in citation networks of scientific publications. *Technovation*, 28(11), 758–775. <https://doi.org/10.1016/J.TECHNOVATION.2008.03.009>
- Sterling, J. A., & Montemore, M. M. (2022). Combining Citation Network Information and Text Similarity for Research Article Recommender Systems. *IEEE Access*, 10, 16–23. <https://doi.org/10.1109/ACCESS.2021.3137960>
- Tang, L., & Liu, H. (2009). Relational Learning via Latent Social Dimensions. *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 817–826. <http://www.techcrunch.com/2008/12/31/top-social->
- Thorup, M. (1999). Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time. *Journal of the ACM*, 46(3), 362–394.
- Van Belle, R., Damme, C. V., Tytgat, H., & De Weerd, J. (2022). Inductive Graph Representation Learning for fraud detection. *Expert Systems With Applications*, 193, Article 116463. <https://doi.org/10.1016/j.eswa.2021.116463>
- Wang, D., Cui, P., & Zhu, W. (2016). Structural Deep Network Embedding. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1225–1234. <https://doi.org/10.1145/2939672.2939753>.
- Wang, H., Wang, J., Wang, J., Zhao, M., Zhang, W., Zhang, F., ... Guo, M. (2018). GraphGAN: Graph representation learning with generative adversarial nets. *Thirty-Second AAAI Conference on Artificial Intelligence*.
- Wu, H., Yi, H., & Li, C. (2021). An integrated approach for detecting and quantifying the topic evolutions of patent technology: A case study on graphene field. *Scientometrics*, 126(8), 6301–6321. <https://doi.org/10.1007/S11192-021-04000-2/FIGURES/3>
- Yang, C., Liu, Z., Zhao, D., Sun, M., & Chang, E. Y. (2015). Network representation learning with rich text information. *Twenty-Fourth International Joint Conference on Artificial Intelligence*.
- Yen, J. Y. (1970). AN ALGORITHM FOR FINDING SHORTEST ROUTES FROM ALL SOURCE NODES TO A GIVEN DESTINATION IN GENERAL NETWORKS. *Quarterly of Applied Mathematics*, 27(4), 526–530.
- Zhang, D., Yin, J., Zhu, X., & Zhang, C. (2018). Network Representation Learning: A Survey. *IEEE Transactions on Big Data*, 6(1), 3–28. <https://doi.org/10.1109/TBDATA.2018.2850013>
- Zhou, H., Zhao, Z., Li, C., Liang, Y., & Zeng, Q. (2019). Rank2vec: Learning node embeddings with local structure and global ranking. *Expert Systems With Applications*, 136, 276–287. <https://doi.org/10.1016/j.eswa.2019.06.045>